Leakage-Free Preprocessing for Word-Level Recitation Error Detection: Strategy Versus Reality

Faruq Hammed
Carnegie Mellon University
fhammed@andrew.cmu.edu

***Abstract*—An offline recitation tutor has to decide, word by word, whether a learner mispronounced a word. We ask a simple question. Does a preprocessing plan, written before any code, still hold once we meet the data? Our plan had two gaps. We ran its audit on all rows, including the test reciters, and it ignored the order of the words. We build the plan as a scikit-learn pipeline that fits every statistic on six training reciters. We repeat the audit on those reciters alone, and we compare 20 model settings with leave-one-reciter-out cross-validation. No planned drop changes. Neighbor features raise out-of-fold PR-AUC from 0.705 to 0.757. The selected model reaches PR-AUC 0.780 (chance 0.168) on two unseen reciters. One feature we built, the length of the edited recording divided by the length of the original, is a shortcut from the data generator. Removing it lowers out-of-fold PR-AUC to 0.595. All errors are synthetic, so we treat 0.780 as an upper bound.**

***Index Terms*—data preprocessing, data leakage, grouped cross-validation, class imbalance, speech alignment**

I. Introduction

A recitation tutor for young learners has to run on a low-end device with no internet. For each word, it must answer one question. Is this word mispronounced? A missed error lets a bad habit form. A false alarm discourages the learner. This report is about the preprocessing that sits in front of that decision.

Before we wrote any code, we fixed a plan. It covered the audit, the dropped columns, the transforms, and the split. This report compares that plan with what we did. Two gaps needed attention. The plan's audit used all 9,086 rows, so the two test reciters could shape the drops. The plan also treated each word on its own, although recitation goes in order.

We claim two things. The plan still holds when we use the training reciters alone, and neighbor features add signal. We test each claim with one change at a time. Our contributions are:

1) A decision log that compares 22 planned items with what happened, each with a metric and a figure (Section III).
2) A pipeline that learns every statistic from training data only, and an audit repeated on the training reciters only. No drop changes (Sections II and V).
3) Evidence that neighbor features raise out-of-fold PR-AUC by 0.052, and that one built feature is a shortcut from the generator (Sections VI and VII).

II. Data and Pipeline

a) Data.: The Suwayhib test set has 2,000 verse recordings from 8 professional reciters. The builder put one error into 1,400 recordings, at a known word. We call that word an error word. A dynamic time warping (DTW) alignment [1] over mel-frequency cepstral coefficients gives one row of features per word. The word table has $N = 9{,}086$ rows and $P = 22$ features. The target `is_error_word` marks 1,400 error words (15.41%). That is a ratio of 5.49:1.

b) Split.: Words from one recording share a voice and a verse, so a split by row would leak. We split by reciter. Six reciters train the model (6,763 rows, 14.9% error words). Two reciters, Husary and Minshawy, test it once (2,323 rows, 16.8% error words). Inside the training side, leave-one-reciter-out cross-validation picks the model.

c) Architecture.: Every step sits inside a scikit-learn [4] `Pipeline` or `ColumnTransformer`. Three custom transformers hold the logic. `FeatureEngineer` adds `rec_len_ratio`, the edited recording length divided by the original length. `IQRWinsorizer` learns its fences on the training data. `NeighborFeatures` adds the `dtw_cost` and `dur_ratio` of the previous word (lag) and the next word (lead) in the same recording. The pipeline imputes with the median and applies $\log(1 + x)$ to four skewed features. It winsorizes at three times the IQR and then standardizes. Each cross-validation fold refits every step on its own training reciters.

III. Strategy Versus Reality

Table I compares each planned item with what we did. Rows 20 to 22 record the three recommendations from the design review.

IV. Evidence for Each Decision

Figures 1.2 and 3.2 (target distribution and collinear scatterplots) are in the repository's `figures/` directory. The other six diagnostic figures appear below.

A. Missingness Mechanism

The column `stretch_rate` is missing for 7,106 of 9,086 words (78.21%). The other 21 feature columns are complete (Fig. 1.1). Little's MCAR test [3] compares missing-data patterns across several incomplete columns. Only one column is incomplete here, so the test is not defined. We test the missingness indicator against the error type instead. The link

TABLE I
DECISION LOG. EACH ROW GIVES THE PLANNED CHOICE, THE EXECUTED CHOICE, AND THE EVIDENCE FOR IT. "FIG." NUMBERS REFER TO SECTION IV.

#	Step	Phase 1 plan	Phase 2 execution	Evidence (metric, figure)
1	Target	Binary, $9{,}086 \times 22$	Built as planned	1,400 error words (15.41%), 5.49:1 (Fig. 1.2)
2	Split	By reciter, 6 train and 2 test	6,763 and 2,323 rows; no shared reciter or recording	Error rate 14.9% against 16.8% (Fig. 1.2)
3	Missingness	MNAR; drop `stretch_rate`	Dropped	78.21% missing; Cramér's $V = 1.000$ with error type (Fig. 1.1)
4	Imputation	Median, as a safety net	In pipeline; nothing left to impute	0 missing values after the pipeline
5	Outlier detection	1.5×IQR rule	As planned	Skew 0.63 to 19.18 on 12 features; $\lvert z\rvert > 3$ flags too few (Fig. 2.1)
6	Outlier treatment	$\log(1+x)$, then winsorize at 3×IQR	0.00% to 2.99% capped on 8 of 9 features; `rec_len_ratio` 8.31%	Fences fit on train (Fig. 2.1)
7	Row deletion	None	9,086 rows in and out	N unchanged
8	Association tests	r_{pb}, Mann–Whitney, MI, χ^2	As planned; MI ranks features	`log_dev` 0.051, `dur_ratio` 0.050 (Fig. 3.3)
9	Leakage drops	Five columns	Fixed list in code	`word_idx` ranks third by MI (0.047), so it carries the generator rule (Fig. 3.3)
10	Collinearity	$\lvert r\rvert \geq 0.85$, VIF > 10	3 planned pairs plus 1 new pair	VIF 16.82 to 2.55 (Figs. 3.1, 3.2)
11	Durations	Replace with `rec_len_ratio`	Kept; MI 0.000 alone	Removing it lowers PR-AUC 0.757 to 0.595 (Section VII)
12	Features	22 to 9	22 to 9	Max VIF 2.55
13	Encoding	None	None	No categorical input remains
14	Scaling	StandardScaler	As planned	Train mean 0, std 1; `width` skew 2.32 to -0.96 (Fig. 4.1)
15	PCA	Diagnostic	Not applied to the model	6 components reach 90%, 7 reach 95% (Fig. 5.1)
16	Validation	Leave-one-reciter-out	As planned, on the full pipeline	6 folds
17	Metrics	Class weights, PR-AUC, F1	Macro F1 added; threshold chosen out of fold	Threshold 0.60; out-of-fold Macro F1 0.820
18	Model	Logistic regression	Gradient boosting with neighbor features	CV PR-AUC 0.463 to 0.757 (Table II)
19	Audit scope	All rows	Repeated on training reciters only	No drop changes (Section V)
20	Review: context	None	Neighbor features added	CV PR-AUC +0.052 (Table III)
21	Review: models	Linear baseline	Random forest, extra trees, LightGBM added	20 candidates (Table II)
22	Review: learners	None	Not done; no learner data exists in the repository	Limitation (Section IX)

is perfect (Cramér's $V = 1.000$, $\chi^2 = 9086$, $\mathrm{df} = 8$, $p < 0.001$). The generator records a stretch only for the two madd error types. The indicator also has a weak link to the target ($V = 0.099$, $p < 0.001$). So the missingness depends on the label we predict. That makes it MNAR [2]. We drop the column and do not impute it. A missingness flag would equal "has a madd error", and a real device never knows that.

B. Outlier Treatment

Some extreme values are real signal here, because a deleted word should give an extreme duration. On all rows, `dur_z` has skew 19.18 (kurtosis 761.5). The feature `suspicion` has skew 18.28 (kurtosis 692.2). The 1.5×IQR fence flags 1,031 values of `dur_z` (11.3%). The rule $\lvert z\rvert > 3$ flags only 108, because heavy tails make the standard deviation large. So we use the IQR rule. After $\log(1+x)$, we winsorize at $Q_1 - 3\,\mathrm{IQR}$ and $Q_3 + 3\,\mathrm{IQR}$. We fit the fences on the training data. Fig. 2.1 shows that this clips only the tails. The cap rate is 0.00% to 2.99% on eight of the nine features. For `cost_z` the fences are ± 4.721, and 2.99% of training values are capped. We cap and do not delete rows. So N stays at 9,086, and each extreme row keeps its other features.

C. Collinearity and Feature Screening

Fig. 3.1 shows four pairs with $\lvert r\rvert \geq 0.85$ on the training split. They are `orig_duration_ms` with `new_duration_ms` ($r = 0.937$), `dur_z` with `suspicion` (0.902), `test_frac` with `ref_frac` (0.860), and `orig_duration_ms` with `n_words` (0.858). The first three pairs match the plan. The fourth is new, but we had already dropped `orig_duration_ms`. Fig. 3.2 shows scatterplots of the three planned pairs. Before the drops, the VIF of `test_frac` is 16.82 and the VIF of `suspicion` is 12.12. We drop `test_frac`, `dur_z`, and both duration columns. Afterward, the VIF of `suspicion` is 1.37, and every VIF is at most 2.55. From each pair we keep the feature

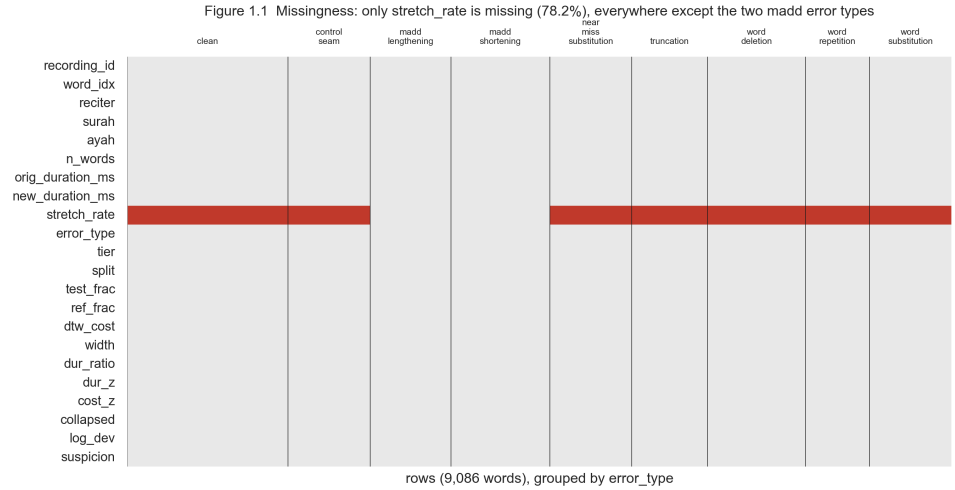


Fig. 1.1. Only `stretch_rate` has missing values. It is present for the two madd error types and absent for the rest.

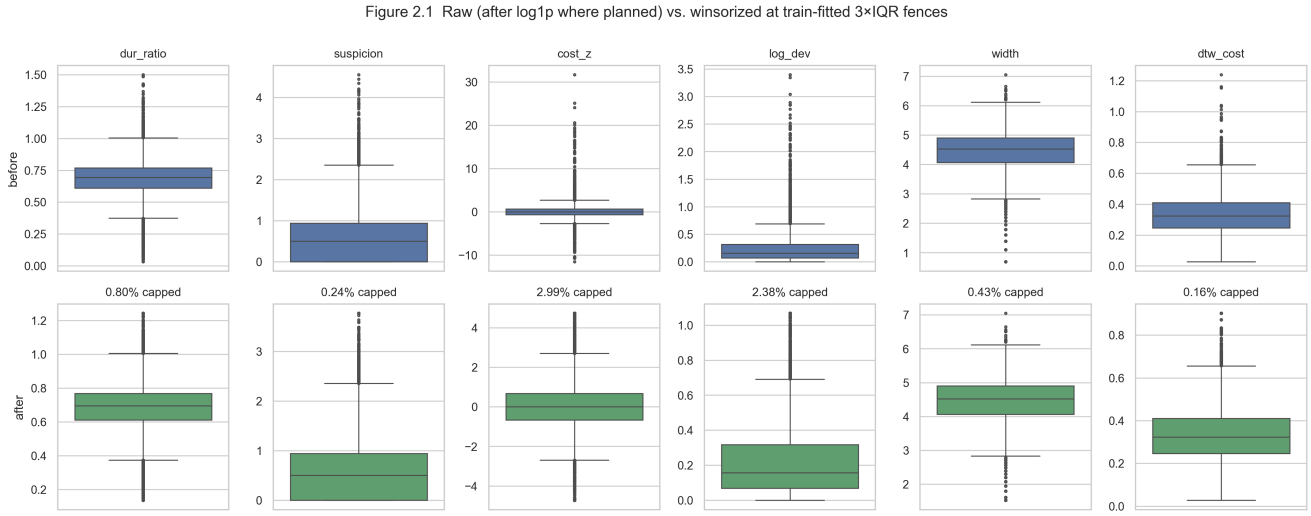


Fig. 2.1. Box plots before (top) and after (bottom) winsorizing at training-fitted 3×IQR fences, for six features. The percentage under each lower plot is the share of training values capped.

with the higher mutual information (MI), as our protocol says (Fig. 3.3). `ref_frac` has MI 0.024 against 0.007 for `test_frac`. `suspicion` has MI 0.044 against 0.040 for `dur_z`. Both duration columns have MI 0.000, so we replace them with their ratio (Section VII).

D. Transformations and Scaling

Fig. 4.1 follows `width` through four stages: raw, $\log(1 + x)$, min–max, and standardized. The log transform lowers the skew from 2.32 to -0.96. The two scalers change only location and scale, so the skew stays at -0.96. After standardization, every training feature has mean 0 and standard deviation 1. The notebook checks this with an assert. We apply the log to `width`, `dur_ratio`, `suspicion`, and `n_words`. For `suspicion`, the skew drops from 18.28 to 1.49. We do not apply it to `rec_len_ratio`. Its skew is -0.76, and the log would make it -1.35. We reject min–max scaling, because one extreme value squeezes the rest of a feature into a narrow band.

E. Variance Cutoffs

We fit PCA to the nine scaled training features, only to look at the variance (Fig. 5.1). The first component explains 27.8% of the variance. The cumulative share is 91.5% at six components and 95.9% at seven. The curve is flat because the collinearity screen already removed the redundant features. We keep the nine named features. Dropping two components saves little, and we would lose the names we need to explain a flagged word.

V. Audit Repeated on Training Reciters

Our plan's audit ran on all rows. That breaks the plan's own rule to split before any fitting. So we repeated the audit on the six training reciters. No drop changes. One log decision

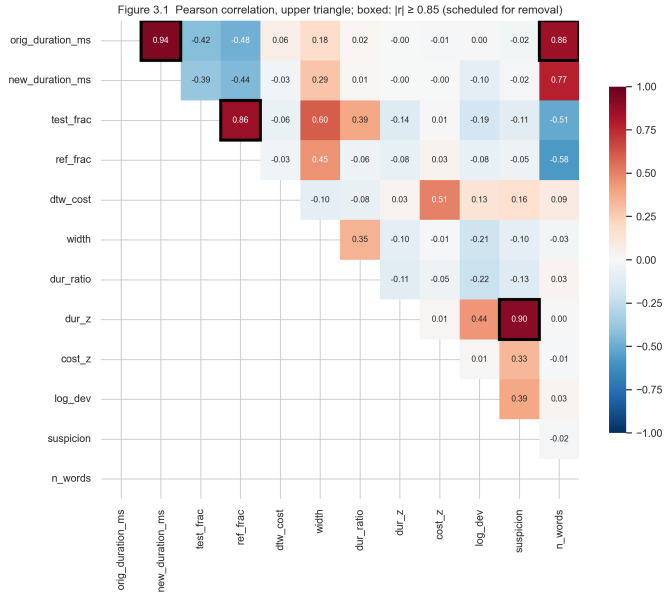


Fig. 3.1. Pearson correlation on the training split, upper triangle. Boxed cells have $|r| \geq 0.85$ and are scheduled for removal.

TABLE II
OUT-OF-FOLD RESULTS ON THE SIX TRAINING RECITERS. "+ NBR" ADDS NEIGHBOR FEATURES. HIGHER IS BETTER FOR BOTH METRICS.

Model	PR-AUC	Std	Macro F1
Logistic regression	0.463	0.039	0.666
Extra trees	0.670	0.019	0.744
Random forest	0.698	0.017	0.787
LightGBM	0.707	0.019	0.774
Gradient boosting	0.705	0.020	0.785
Extra trees + nbr	0.699	0.029	0.762
Random forest + nbr	0.743	0.020	0.804
LightGBM + nbr	0.748	0.020	0.795
Gradient boosting + nbr	**0.757**	0.029	**0.812**

TABLE III
ABLATIONS OF THE BEST MODEL ON THE SAME FOLDS. HIGHER IS BETTER.

Variant	PR-AUC	Macro F1
As selected (13 features)	0.757	0.812
No neighbor features	0.705	0.785
No `rec_len_ratio`	0.595	0.751
Add `word_idx` back	0.773	0.821

becomes a judgment call. The skew of `dur_ratio` is 1.22 on all rows and 0.92 on the training reciters. That is just under our rule of skew above 1. We keep the log, because 0.92 is still moderate. The skew of `suspicion` is 8.70 on the training reciters and 18.28 on all rows, because a few extreme values come from the test reciters. The MI ranking shifts a little (`log_dev` 0.051, `dur_ratio` 0.050, `word_idx` third). The top five features stay the same. One concern remains upstream. The features `suspicion`, the z-scores, and `dtw_cost` come from a scorer outside our repository. We cannot rule out that someone tuned it on these reciters.

VI. Model Comparison

Gradient boosting with neighbor features has the best out-of-fold PR-AUC (0.757). We ran leave-one-reciter-out cross-validation on the six training reciters ($n = 6{,}763$ words, chance PR-AUC 0.149). We select by PR-AUC, because it stays informative under a 5.49:1 imbalance [6]. Table II lists the best setting of each family. LightGBM [5] is one of the five families. Logistic regression reaches 0.463. The tree ensembles gain about 0.25 PR-AUC, because `dur_ratio` carries signal that is not linear (high MI, $|r_{pb}| = 0.08$). Without neighbor features, gradient boosting, LightGBM, and random forest are within 0.01 of each other (0.698 to 0.707). That is less than one fold standard deviation. Extra trees are the weakest tree model in every setting. The penalty strength of logistic regression has no effect. $C = 1$ and $C = 0.1$ match to three decimals, and the fitted coefficients differ by at most 0.01.

a) Neighbor features help every tree model.: Gradient boosting rises from 0.705 to 0.757. Random forest rises from 0.698 to 0.743, and LightGBM from 0.707 to 0.748. Logistic regression barely moves (0.463 to 0.466). So the gain comes from interactions. The first and last word of a recording have no neighbor on one side. We fill the gap with the word's own value. A median or a flag would show that the word is first, which is the `word_idx` rule we removed.

b) Adding `word_idx` back raises PR-AUC by 0.016.: That gain is the shortcut we removed. The generator never puts an error in the first word, and the feature lets the model learn that rule. It stays out. So the 14 dropped columns do hold information about this dataset. They do not hold information that would carry over to real learners.

VII. A Shortcut From the Generator

Alone, `rec_len_ratio` carries almost no signal (MI $=$ 0.000, $r_{pb} = -0.028$ on the training split). Together with other features it matters a lot. Removing it lowers out-of-fold PR-AUC from 0.757 to 0.595 (Table III). A tree can combine "this recording is shorter than its original" with a per-word duration signal. Then it finds the edited word. The reason is in how the builder made the data. The ratio is exactly 1.000 for all 400 clean recordings (std 0.000). It is 0.992 for control seams, 0.766 for deletions, 0.528 for truncations, and 1.216 for repetitions. A value of 1.000 means the recording is untouched. That is the same kind of shortcut as `stretch_rate`. A device could measure a length ratio against a reference. But a learner's length differs from any reference for normal reasons, so "exactly 1.0 when clean" will not hold. We keep the feature, because our plan includes it and a device can measure it. We report results with and without it. We treat the numbers with it as an upper bound.

VIII. Held-Out Test and Verification

a) The selected model reaches PR-AUC 0.780 on the two test reciters.: We refit on all six training reciters and

Figure 3.3 Feature-to-target association, ranked by mutual information

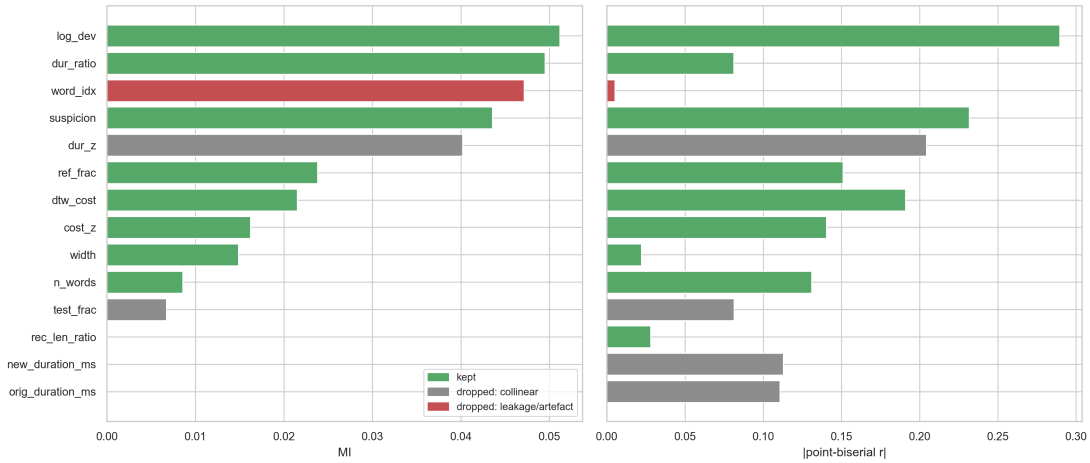


Fig. 3.3. Mutual information and $|r_{pb}|$ with the target, training split. The generator feature `word_idx` ranks third.

Figure 4.1 Transformation stages for 'width' (training split)

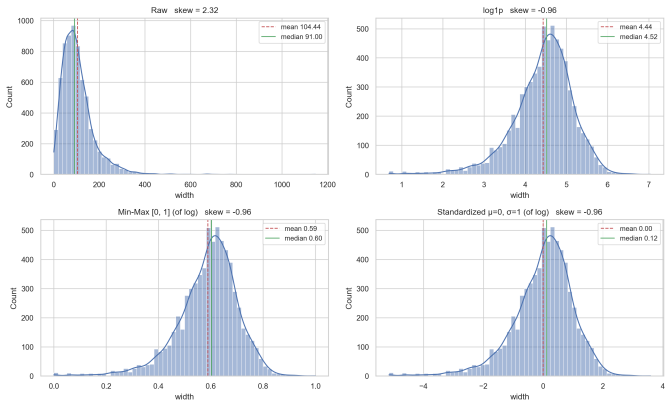


Fig. 4.1. The feature `width` at four stages. The log transform changes the shape (skew 2.32 to -0.96). The scalers change only location and scale.

Figure 5.1 PCA scree plot with 90% / 95% cumulative variance cutoffs

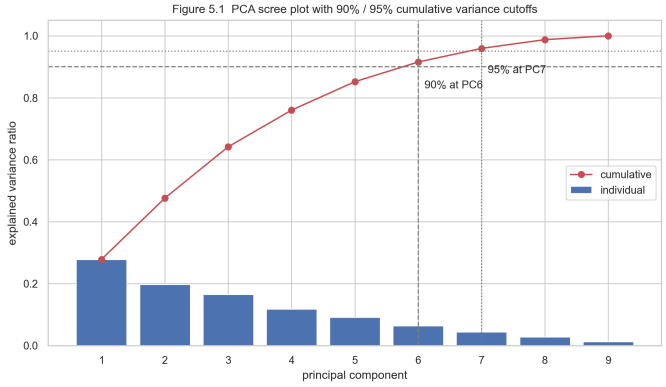


Fig. 5.1. Explained variance per component (bars) and cumulative variance (line), with 90% and 95% cutoffs.

TABLE IV
HELD-OUT TEST RESULT AT THRESHOLD 0.60. HIGHER IS BETTER.

PR-AUC	ROC-AUC	Macro F1	F1 (error)	Prec.	Recall
0.780	0.924	0.829	0.712	0.759	0.670

scored the test reciters once ($n = 2{,}323$ words, 391 error words, chance PR-AUC 0.168). We chose the threshold out of fold. The value 0.60 gives the best Macro F1 on the training predictions (0.820). Table IV has the result. The threshold favors precision over recall. That suits the device, because false alarms discourage learners. The test score is 0.023 above the cross-validation score, so we see no sign of leakage. Two reciters can produce a gap like that by chance.

b) Pipeline output verification.: Table V confirms the matrix sizes. The preprocessor cuts 22 features to 9 and deletes no rows. We chose the 14 dropped columns to remove leakage and overlap. Five are leakage or generator columns (`error_type`, `tier`, `stretch_rate`, `split`, `word_idx`). Four are identifiers (`recording_id`, `reciter`, `surah`, `ayah`). One has near-zero variance (`collapsed`). Four repeat a feature we keep (`test_frac`, `dur_z`, and the two durations). Only `word_idx` costs measurable performance, and we drop it on purpose.

TABLE V
OUTPUT VERIFICATION. THE NOTEBOOK ASSERTS EACH CHECK.

Check	Result
Raw matrix ($N \times P$)	$9{,}086 \times 22$
Preprocessed train and test	$6{,}763 \times 9$ and $2{,}323 \times 9$
Final model inputs	13 (9 plus 4 neighbor)
Rows lost	0
Missing values after the pipeline	0
Train mean and std	0 and 1 per feature
Shared reciters or recordings	none
Maximum VIF	2.55

IX. Limitations and Next Steps

Every error is a synthetic splice into the audio of a professional reciter. The data has no real learners and no Hausa-accented or Fulfulde-accented speech. So 0.780 is an upper bound. For this dataset, the honest range in cross-validation is 0.595 without `rec_len_ratio` to 0.757 with it. Eight reciters are few voices, and the test set has only two. The generator never puts two errors in one recording, or an error in the first word. A learner can do both. We did not build the sequence model that the review recommended (a 1D convolutional network or a bidirectional LSTM). It needs a library that the project does not use. The threshold of 0.60 is one operating point chosen for Macro F1. We did not choose it for the cost of a false alarm. Our next step is to score real learner recordings, for example from IqraEval, with the same alignment features.

References

[1] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 26, no. 1, pp. 43–49, 1978.

[2] D. B. Rubin, "Inference and missing data," *Biometrika*, vol. 63, no. 3, pp. 581–592, 1976.

[3] R. J. A. Little, "A test of missing completely at random for multivariate data with missing values," *J. Amer. Statist. Assoc.*, vol. 83, no. 404, pp. 1198–1202, 1988.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 3146–3154.

[6] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. Int. Conf. Mach. Learn.*, 2006, pp. 233–240.